



Работа с файлами

Открытие и закрытие файлов

Последнее обновление: 02.02.2024



Python поддерживает множество различных типов файлов, но условно их можно разделить на два вида: текстовые и бинарные. Текстовые файлы - это к примеру файлы с расширением cvs, txt, html, в общем любые файлы, которые сохраняют информацию в текстовом виде. Бинарные файлы - это изображения, аудио и видеофайлы и т.д. В зависимости от типа файла работа с ним может немного отличаться.

При работе с файлами необходимо соблюдать некоторую последовательность операций:

1. Открытие файла с помощью метода **open()**
2. Чтение файла с помощью метода **read()** или запись в файл посредством метода **write()**
3. Закрытие файла методом **close()**

Открытие и закрытие файла

Чтобы начать работу с файлом, его надо открыть с помощью функции **open()**, которая имеет следующее формальное определение:

```
1 | open(file, mode)
```

Первый параметр функции представляет путь к файлу. Путь файла может быть абсолютным, то есть начинаться с буквы диска, например, *C://somedir/somefile.txt*. Либо можно быть относительным, например, *somedir/somefile.txt* - в этом случае поиск файла будет идти относительно расположения запущенного скрипта Python.

Второй передаваемый аргумент - `mode` устанавливает режим открытия файла в зависимости от того, что мы собираемся с ним делать. Существует 4 общих режима:

- **r** (Read). Файл открывается для чтения. Если файл не найден, то генерируется исключение `FileNotFoundError`
- **w** (Write). Файл открывается для записи. Если файл отсутствует, то он создается. Если подобный файл уже есть, то он создается заново, и соответственно старые данные в нем стираются.
- **a** (Append). Файл открывается для дозаписи. Если файл отсутствует, то он создается. Если подобный файл уже есть, то данные записываются в его конец.
- **b** (Binary). Используется для работы с бинарными файлами. Применяется вместе с другими режимами - `w` или `r`, например, **rb** (чтение бинарных файлов) и **wb** (запись бинарных файлов).
- **r+**. Файл открывается одновременно для чтения и записи. Если файл не найден, то генерируется исключение `FileNotFoundError`
- **w+**. Файл открывается одновременно для чтения и записи. Если файл не существует, то он автоматически создается. Если файл существует, то он перезаписывается
- **a+**. Файл открывается одновременно для чтения и записи. Если файл не существует, то он автоматически создается. Если файл существует, то данные добавляются в конец файла

Например, открытие текстового файла на запись

```
1 myfile = open("hello.txt", "w")
```

Или открытие бинарного файла на чтение

```
1 myfile = open("image.png", "rb")
```

После завершения работы с файлом его обязательно нужно закрыть методом `close()`. Данный метод освободит все связанные с файлом используемые ресурсы.

Например, откроем для записи текстовый файл "hello.txt":

```
1 myfile = open("hello.txt", "w")
2
3 myfile.close()
```

При открытии файла или в процессе работы с ним мы можем столкнуться с различными исключениями, например, к нему нет доступа и т.д. В этом случае программа выпадет в ошибку, а ее выполнение не дойдет до вызова метода `close`, и соответственно файл не будет закрыт.

В этом случае мы можем обрабатывать исключения:

```
1 try:
2     myfile = open("hello.txt", "w")
3     try:
4         print("Работа с файлом")
5     finally:
6         myfile.close()
7 except Exception as ex:
8     print(ex)
```

В данном случае вся работа с файлом идет во вложенном блоке try. И если вдруг возникнет какое-либо исключение, то в любом случае в блоке finally файл будет закрыт.

Однако есть и более удобная конструкция - конструкция **with**:

```
1 with open(file, mode) as myfile:
2     инструкции
```

Эта конструкция определяет для открытого файла переменную myfile и выполняет набор инструкций. После их выполнения файл автоматически закрывается. Даже если при выполнении инструкций в блоке with возникнут какие-либо исключения, то файл все равно закрывается.

Так, перепишем предыдущий пример:

```
1 with open("hello.txt", "w") as myfile:
2     print("Работа с файлом myfile")
```

[Назад](#) [Содержание](#) [Вперед](#)



Помощь сайту

[Помощь сайту](#)

Юмани:

410011174743222

Номер карты:

4048415020898850

[Телеграмм](#)

[Вконтакте](#) | [Телеграм](#) | [Донаты/Помощь сайту](#)

Contacts: metanit22@mail.ru

Copyright © Евгений Попов, metanit.com, 2025. Все права защищены.



Текстовые файлы

Последнее обновление: 02.02.2024



Запись в текстовый файл

Чтобы открыть текстовый файл на запись, необходимо применить режим **w** (перезапись), **a** (дозапись) или **r+ / w+** (чтение и запись). Затем для записи применяется метод **write(str)**, в который передается записываемая строка. Стоит отметить, что записывается именно строка, поэтому, если нужно записать числа, данные других типов, то их предварительно нужно конвертировать в строку.

Запишем некоторую информацию в файл "hello.txt":

```
1 with open("hello.txt", "w") as file:
2     file.write("hello world")
3
4 print("Файл записан")
```

Если мы откроем папку, в которой находится текущий скрипт Python, то увидим там файл "hello.txt". Этот файл можно открыть в любом текстовом редакторе и при желании изменить.

Теперь дозапишем в этот файл еще одну строку:

```
1 with open("hello.txt", "a") as file:
```

```
2     file.write("\nhello work")
3
4     print("Файл изменен")
```

Дозапись выглядит как добавление строки к последнему символу в файле, поэтому, если необходимо сделать запись с новой строки, то можно использовать эскейп-последовательность `"\n"`. В итоге файл `hello.txt` будет иметь следующее содержимое:

```
1 hello world
2 hello work
```

Для записи списка строк имеется метод **writelines()**. В качестве параметра он принимает список строк:

```
1 lines = ["Hello Word\n", "Hello Work\n", "Hello World\n"]
2 with open("hello2.txt", "a") as file:
3     file.writelines(lines)
4
5     print("Список строк записан в файл")
```

Стоит учитывать, что, если мы хотим, чтобы каждая строка из списка располагалась на отдельной строке в файле, то строка в списке должна заканчиваться на символ `"\n"`.

Еще один способ записи в файл представляет стандартный метод **print()**, который применяется для вывода данных на консоль:

```
1 with open("hello.txt", "a") as myfile:
2     print("\nhello metanit.com", file=myfile)
```

Для вывода данных в файл в метод `print` в качестве второго параметра передается название файла через параметр `file`. А первый параметр представляет записываемую в файл строку.

Чтение файла

Для чтения файла он открывается с режимом **r** (Read), и затем мы можем считать его содержимое различными методами:

- **readline()**: считывает одну строку из файла
- **read()**: считывает все содержимое файла в одну строку
- **readlines()**: считывает все строки файла в список

Например, считаем выше записанный файл построчно:

```
1 with open("hello.txt", "r") as file:
2     for line in file:
3         print(line)
```

Несмотря на то, что мы явно не применяем метод `readline()` для чтения каждой строки, но в при переборе файла этот метод автоматически вызывается для получения каждой новой строки. Поэтому в цикле вручную нет смысла вызывать метод `readline`. Консольный вывод:

```
hello world

hello work

hello metanit.com
```

Однако поскольку функция `print` неявно добавляет к каждой строке перевод строки `"\n"`, а строки в записанном файлетакже разделяются символом перевода строки `"\n"`, то получается, что каждая строка оканчивается двойным `"\n"`. и чтобы исключить излишнего переноса на другую строку в функцию `print` можно передать значение `end=""`.

```
1 with open("hello.txt", "r") as file:
2     for line in file:
3         print(line, end="")
```

Теперь явным образом вызовем метод `readline()` для чтения отдельных строк:

```
1 with open("hello.txt", "r") as file:
2     str1 = file.readline()      # считываем первую строку
3     print(str1, end="")
4     str2 = file.readline()      # считываем вторую строку
5     print(str2)
```

Консольный вывод:

```
hello world
hello work
```

Метод `readline` можно использовать для построчного считывания файла в цикле `while`:

```
1 with open("hello.txt", "r") as file:
2     line = file.readline()
3     while line:
4         print(line, end="")
5         line = file.readline()
```

Если файл небольшой, то его можно разом считать с помощью метода `read()`:

```
1 with open("hello.txt", "r") as file:
2     content = file.read()
3     print(content)
```

И также применим метод `readlines()` для считывания всего файла в список строк:

```
1 with open("hello.txt", "r") as file:
2     contents = file.readlines()
3     str1 = contents[0]
4     str2 = contents[1]
5     print(str1, end="")
6     print(str2)
```

При чтении файла мы можем столкнуться с тем, что его кодировка не совпадает с ASCII. В этом случае мы явным образом можем указать кодировку с помощью параметра `encoding`:

```
1 filename = "hello.txt"
2 with open(filename, encoding="utf8") as file:
3     text = file.read()
```

Чтение и запись

Для открытия файла сразу для чтения и записи применяются флаги:

- **r+**: если файл не найден, то генерируется исключение `FileNotFoundError`
- **w+**: если файл не существует, то он автоматически создается. Если файл существует, то он перезаписывается

Например, запишем и считаем из файла данные:

```
1 with open("hello.txt", "w+") as file:
2     file.write("Hello world\nHello work\n") # сначала записываем данные
3     file.seek(0) # перемещаемся к первому байту в файле
4     content = file.read() # считываем данные
5     print(content)
```

Перемещение по файлу

Для перемещения по файлу после его открытия применяется метод **seek()**. В него передается индекс байта, к которому надо переместить указатель в файле. Например, в примере выше после записи указатель в файле автоматически устанавливается на конец файла. И считывания данных идет с позиции, на которой стоит указатель. И чтобы его сдвинуть в самое начало, применяется вызов

```
1 file.seek(0)
```

Аналогичным образом мы можем перемещаться по файлу на нужную позицию. Например, считаем данные, начиная с 5-го символа:

```
1 with open("hello.txt", "w+") as file:
2     file.write("Hello world\n") # сначала записываем данные
3     file.seek(6)                # перемещаемся к шестому байту в файле
4     content = file.read()        # считываем данные
5     print(content)              # world
```

Пример работы с файлами

Теперь напишем небольшой скрипт, в котором будет записывать введенный пользователем массив строк и считывать его обратно из файла на консоль:

```
1 # имя файла
2 FILENAME = "messages.txt"
3
4 # запись строки в файл
5 def write():
6     message = input("Введите строку: ")
7     with open(FILENAME, "a") as file:
8         file.write(message + "\n")
9
10 # чтение файла файл
11 def read():
12     with open(FILENAME, "r") as file:
13         for message in file:
14             print(message, end="")
15     print() # перевод строки для разделения меню и вывода
16
17 while(True):
18     selection = int(input("1.Запись в файл\t\t2.Чтение файла\t\t3.Выход\nВыберите д
19     match selection:
20         case 1: write()
21         case 2: read()
22         case 3: break
23         case _: print("Некорректный ввод")
24
25 print("Программа завершена")
```

Пример работы программы:

```
1.Запись в файл          2.Чтение файла          3.Выход
Выберите действие: 1
Введите строку: Hello World

1.Запись в файл          2.Чтение файла          3.Выход
Выберите действие: 1
Введите строку: Hello METANIT.COM

1.Запись в файл          2.Чтение файла          3.Выход
Выберите действие: 2
Hello World
Hello METANIT.COM

1.Запись в файл          2.Чтение файла          3.Выход
Выберите действие: 3
Программа завершена
```

[Назад](#) [Содержание](#) [Вперед](#)



Помощь сайту

[Помощь сайту](#)

Юмани:

410011174743222

Номер карты:

4048415020898850

[Телеграмм](#)

[Вконтакте](#) | [Телеграм](#) | [Донаты/Помощь сайту](#)

Contacts: metanit22@mail.ru

Copyright © Евгений Попов, metanit.com, 2025. Все права защищены.



Файлы CSV

Последнее обновление: 02.02.2024



Одним из распространенных файловых форматов, которые хранят в удобном виде информацию, является формат **csv**. Каждая строка в файле csv представляет отдельную запись или строку, которая состоит из отдельных столбцов, разделенных запятыми. Собственно поэтому формат и называется Comma Separated Values. Но хотя формат csv - это формат текстовых файлов, Python для упрощения работы с ним предоставляет специальный встроенный модуль **csv**.

Рассмотрим работу модуля на примере:

```
1 import csv
2
3 FILENAME = "users.csv"
4
5 users = [
6     ["Tom", 28],
7     ["Alice", 23],
8     ["Bob", 34]
9 ]
10
11 with open(FILENAME, "w", newline="") as file:
12     writer = csv.writer(file)
13     writer.writerows(users)
14
15
16 with open(FILENAME, "a", newline="") as file:
17     user = ["Sam", 31]
18     writer = csv.writer(file)
19     writer.writerow(user)
```

В файл записывается двухмерный список - фактически таблица, где каждая строка представляет одного пользователя. А каждый пользователь содержит два поля - имя и возраст. То есть фактически таблица из трех строк и двух столбцов.

При открытии файла на запись в качестве третьего параметра указывается значение `newline=""` - пустая строка позволяет корректно считывать строки из файла вне зависимости от операционной системы.

Для записи нам надо получить объект **writer**, который возвращается функцией `csv.writer(file)`. В эту функцию передается открытый файл. А собственно запись производится с помощью метода `writer.writerows(users)`. Этот метод принимает набор строк. В нашем случае это двумерный список.

Если необходимо добавить одну запись, которая представляет собой одномерный список, например, `["Sam", 31]`, то в этом случае можно вызвать метод **writer.writerow(user)**.

В итоге после выполнения скрипта в той же папке окажется файл `users.csv`, который будет иметь следующее содержимое:

```
1 Tom,28
2 Alice,23
3 Bob,34
4 Sam,31
```

Для чтения из файла нам наоборот нужно создать объект **reader**:

```
1 import csv
2
3 FILENAME = "users.csv"
4
5 with open(FILENAME, "r", newline="") as file:
6     reader = csv.reader(file)
7     for row in reader:
8         print(row[0], " - ", row[1])
```

При получении объекта `reader` мы можем в цикле перебрать все его строки:

```
Tom - 28
Alice - 23
Bob - 34
Sam - 31
```

Работа со словарями

В примере выше каждая запись или строка представляла собой отдельный список, например, `["Sam", 31]`. Но кроме того, модуль `csv` имеет специальные дополнительные возможности для работы со словарями. В частности, функция **csv.DictWriter()** возвращает объект `writer`, который позволяет записывать в файл. А функция **csv.DictReader()** возвращает объект `reader` для чтения из файла. Например:

```
1 import csv
2
3 FILENAME = "users2.csv"
4
5 users = [
6     {"name": "Tom", "age": 28},
7     {"name": "Alice", "age": 23},
8     {"name": "Bob", "age": 34}
9 ]
10
11 with open(FILENAME, "w", newline="") as file:
12     columns = ["name", "age"]
13     writer = csv.DictWriter(file, fieldnames=columns)
14     writer.writeheader()
15
16     # запись нескольких строк
17     writer.writerows(users)
18
19     user = {"name" : "Sam", "age": 41}
20     # запись одной строки
21     writer.writerow(user)
22
23 with open(FILENAME, "r", newline="") as file:
24     reader = csv.DictReader(file)
25     for row in reader:
26         print(row["name"], "-", row["age"])
```

Запись строк также производится с помощью методов `writerow()` и `writerows()`. Но теперь каждая строка представляет собой отдельный словарь, и кроме того, производится запись и заголовков столбцов с помощью метода **`writeheader()`**, а в метод `csv.DictWriter` в качестве второго параметра передается набор столбцов.

При чтении строк, используя названия столбцов, мы можем обратиться к отдельным значениям внутри строки: `row["name"]`.

[Назад](#) [Содержание](#) [Вперед](#)



Помощь сайту

[Помощь сайту](#)

Юмани:

410011174743222

Номер карты:

4048415020898850

[Телеграмм](#)

[Вконтакте](#) | [Телеграм](#) | [Донаты/Помощь сайту](#).

Contacts: metanit22@mail.ru

Copyright © Евгений Попов, metanit.com, 2025. Все права защищены.



Бинарные файлы

Последнее обновление: 02.02.2024



Бинарные файлы в отличие от текстовых хранят информацию в виде набора байт. При открытии бинарного файла на чтение или запись также надо учитывать, что нам нужно применять режим "b" в дополнение к режиму записи ("w") или чтения ("r").

При чтении мы получаем файл в виде набора байт, и наоборот, при записи в метод `write()` передается набор байт. Например, скопируем файл:

```
1 FILENAME = "forest.png"           # файл для чтения
2 NEWFILENAME = "forest_new.png"    # файл для записи
3
4 image_data = []                  # список для хранения считанных данных
5
6 # считываем файл в список image_data
7 with open(FILENAME, "rb") as file:
8     image_data = file.read()
9
10 # запись выше считанных байт в новый файл
11 with open(NEWFILENAME, "wb") as file:
12     file.write(image_data)
13
14 print(f"Файл {FILENAME} скопирован в {NEWFILENAME}")
```

Считывает файл, путь к которому хранится в переменной `FILENAME`. В данном случае это файл изображения "forest.png". Считанные байты помещаются в список `image_data`. Затем этот список записываем в файл с именем `NEWFILENAME`. Таким образом, мы скопируем содержимое одного файла в другой.

Модуль `pickle`

Также для работы с бинарными файлами Python предоставляет специальный встроенный модуль **`pickle`**, который упрощает работу с бинарными файлами. Этот модуль предоставляет два метода:

- **dump(obj, file):** записывает объект obj в бинарный файл file
- **load(file):** считывает данные из бинарного файла в объект

Допустим, надо надо сохранить значения двух переменных:

```
1 import pickle
2
3 FILENAME = "user.dat"
4
5 name = "Tom"
6 age = 19
7
8 with open(FILENAME, "wb") as file:
9     pickle.dump(name, file)
10    pickle.dump(age, file)
11
12 with open(FILENAME, "rb") as file:
13     name = pickle.load(file)
14     age = pickle.load(file)
15     print("Имя:", name, "\tВозраст:", age)
```

С помощью функции dump последовательно записываются два объекта. Поэтому при чтении файла также последовательно посредством функции load мы можем считать эти объекты. Консольный вывод программы:

```
Имя: Том          Возраст: 28
```

Подобным образом мы можем сохранять и извлекать из файла наборы объектов:

```
1 import pickle
2
3 FILENAME = "users.dat"
4
5 users = [
6     ["Tom", 28, True],
7     ["Alice", 23, False],
8     ["Bob", 34, False]
9 ]
10
11 with open(FILENAME, "wb") as file:
12     pickle.dump(users, file)
13
14
15 with open(FILENAME, "rb") as file:
16     users_from_file = pickle.load(file)
17     for user in users_from_file:
18         print("Имя:", user[0], "\tВозраст:", user[1], "\tЖенат/замужем:", user[2])
```

В зависимости от того, какой объект мы записывали функцией `dump`, тот же объект будет возвращен функцией `load` при считывании файла.

Консольный вывод:

Имя: Tom	Возраст: 28	Женат/замужем: True
Имя: Alice	Возраст: 23	Женат/замужем: False
Имя: Bob	Возраст: 34	Женат/замужем: False

[Назад](#) [Содержание](#) [Вперед](#)



Помощь сайту

[Помощь сайту](#)

Юмани:

410011174743222

Номер карты:

4048415020898850

[Телеграмм](#)

[Вконтакте](#) | [Телеграм](#) | [Донаты/Помощь сайту](#)

Contacts: metanit22@mail.ru

Copyright © Евгений Попов, metanit.com, 2025. Все права защищены.



Модуль shelve

Последнее обновление: 02.02.2024



Для работы с бинарными файлами в Python может применяться еще один модуль - **shelve**. Он сохраняет объекты в файл с определенным ключом. Затем по этому ключу может извлечь ранее сохраненный объект из файла. Процесс работы с данными через модуль shelve напоминает работу со словарями, которые также используют ключи для сохранения и извлечения объектов.

Для открытия файла модуль shelve использует функцию **open()**:

```
1 open(путь_к_файлу[, flag="c"[, protocol=None[, writeback=False]]])
```

Где параметр flag может принимать значения:

- **c**: файл открывается для чтения и записи (значение по умолчанию). Если файл не существует, то он создается.
- **r**: файл открывается только для чтения.
- **w**: файл открывается для записи.
- **n**: файл открывается для записи. Если файл не существует, то он создается. Если он существует, то он перезаписывается.

Для закрытия подключения к файлу вызывается метод **close()**:

```
1 import shelve
2 d = shelve.open(filename)
3 d.close()
```

Либо можно открывать файл с помощью оператора **with**. Сохраним и считаем в файл несколько объектов:

```
1 import shelve
2
3 FILENAME = "states2"
```

```
4 with shelve.open(FILENAME) as states:
5     states["London"] = "Great Britain"
6     states["Paris"] = "France"
7     states["Berlin"] = "Germany"
8     states["Madrid"] = "Spain"
9
10 with shelve.open(FILENAME) as states:
11     print(states["London"])
12     print(states["Madrid"])
```

Запись данных предполагает установку значения для определенного ключа:

```
1 states["London"] = "Great Britain"
```

А чтение из файла эквивалентно получению значения по ключу:

```
1 print(states["London"])
```

В качестве ключей используются строковые значения. Консольный вывод программы:

```
Great Britain
Spain
```

Чтение данных

При чтении данных, если запрашиваемый ключ отсутствует, то генерируется исключение. В этом случае перед получением мы можем проверять на наличие ключа с помощью оператора **in**:

```
1 with shelve.open(FILENAME) as states:
2     key = "Brussels"
3     if key in states:
4         print(states[key])
```

Также мы можем использовать метод **get()**. Первый параметр метода - ключ, по которому следует получить значение, а второй - значение по умолчанию, которое возвращается, если ключ не найден.

```
1 with shelve.open(FILENAME) as states:
2     state = states.get("Brussels", "Undefined")
3     print(state)
```

Используя цикл **for**, можно перебрать все значения из файла:

```
1 with shelve.open(FILENAME) as states:
2     for key in states:
3         print(key, " - ", states[key])
```

Метод **keys()** возвращает все ключи из файла, а метод **values()** - все значения:

```
1 with shelve.open(FILENAME) as states:
2
3     for city in states.keys():
4         print(city, end=" ")      # London Paris Berlin Madrid
5     print()
6     for country in states.values():
7         print(country, end=" ")   # Great Britain France Germany Spain
```

Еще один метод **items()** возвращает набор кортежей. Каждый кортеж содержит ключ и значение.

```
1 with shelve.open(FILENAME) as states:
2
3     for state in states.items():
4         print(state)
```

Консольный вывод:

```
("London", "Great Britain")
("Paris", "France")
("Berlin", "Germany")
("Madrid", "Spain")
```

Обновление данных

Для изменения данных достаточно присвоить по ключу новое значение, а для добавления данных - определить новый ключ:

```
1 import shelve
2
3 FILENAME = "states2"
4 with shelve.open(FILENAME) as states:
5     states["London"] = "Great Britain"
6     states["Paris"] = "France"
7     states["Berlin"] = "Germany"
8     states["Madrid"] = "Spain"
9
10 with shelve.open(FILENAME) as states:
11
12     states["London"] = "United Kingdom"
13     states["Brussels"] = "Belgium"
14     for key in states:
15         print(key, " - ", states[key])
```

Удаление данных

Для удаления с одновременным получением можно использовать функцию **pop()**, в которую передается ключ элемента и значение по умолчанию, если ключ не найден:

```
1 with shelve.open(FILENAME) as states:
2
3     state = states.pop("London", "NotFound")
4     print(state)
```

Также для удаления может применяться оператор **del**:

```
1 with shelve.open(FILENAME) as states:
2
3     del states["Madrid"]    # удаляем объект с ключом Madrid
```

Для удаления всех элементов можно использовать метод **clear()**:

```
1 with shelve.open(FILENAME) as states:
2
3     states.clear()
```

[Назад](#) [Содержание](#) [Вперед](#)



Помощь сайту

[Помощь сайту](#)

Юмани:

410011174743222

Номер карты:

4048415020898850

[Телеграмм](#)

[Вконтакте](#) | [Телеграм](#) | [Донаты/Помощь сайту](#)

Contacts: metanit22@mail.ru

Copyright © Евгений Попов, metanit.com, 2025. Все права защищены.



Модуль OS и работа с файловой системой

Последнее обновление: 02.02.2024



Ряд возможностей по работе с каталогами и файлами предоставляет встроенный модуль **os**. Хотя он содержит много функций, рассмотрим только основные из них:

- **mkdir()**: создает новую папку
- **rmdir()**: удаляет папку
- **rename()**: переименовывает файл
- **remove()**: удаляет файл

Создание и удаление папки

Для создания папки применяется функция **mkdir()**, в которую передается путь к создаваемой папке:

```
1 import os
2
3 # путь относительно текущего скрипта
4 os.mkdir("hello")
5 # абсолютный путь
6 os.mkdir("c://somedir")
7 os.mkdir("c://somedir/hello")
```

Для удаления папки используется функция **rmdir()**, в которую передается путь к удаляемой папке:

```
1 import os
2
3 # путь относительно текущего скрипта
4 os.rmdir("hello")
5 # абсолютный путь
6 os.rmdir("c://somedir/hello")
```

Переименование файла

Для переименования вызывается функция **rename(source, target)**, первый параметр которой - путь к исходному файлу, а второй - новое имя файла. В качестве путей могут использоваться как абсолютные, так и относительные. Например, пусть в папке *C://SomeDir/* располагается файл *somefile.txt*. Переименуем его в файл "hello.txt":

```
1 import os
2
3 os.rename("C://SomeDir/somefile.txt", "C://SomeDir/hello.txt")
```

Удаление файла

Для удаления вызывается функция **remove()**, в которую передается путь к файлу:

```
1 import os
2
3 os.remove("C://SomeDir/hello.txt")
```

Существование файла

Если мы попытаемся открыть файл, который не существует, то Python выбросит исключение `FileNotFoundError`. Для отлова исключения мы можем использовать конструкцию `try...except`. Однако можно уже до открытия файла проверить, существует ли он или нет с помощью метода **os.path.exists(path)**. В этот метод передается путь, который необходимо проверить:

```
1 filename = input("Введите путь к файлу: ")
2 if os.path.exists(filename):
3     print("Указанный файл существует")
4 else:
5     print("Файл не существует")
```

[Назад](#) [Содержание](#) [Вперед](#)



Помощь сайту

[Помощь сайту](#)

Юмани:

410011174743222

Номер карты:

4048415020898850

[Телеграмм](#)

[Вконтакте](#) | [Телеграм](#) | [Донаты/Помощь сайту](#).

Contacts: metanit22@mail.ru

Copyright © Евгений Попов, metanit.com, 2025. Все права защищены.



Программа подсчета слов

Последнее обновление: 02.02.2024



Рассмотрим работу со строками на небольшом примере, который будет представлять программу подсчета слов.

Пусть весь код программы будет выглядеть следующим образом:

```
1  # Программа подсчета слов в файле
2  import os
3
4  def get_words(filename):
5      with open(filename, encoding="utf8") as file:
6          text = file.read()
7          text = text.replace("\n", " ")
8          text = text.replace(",", "").replace(".", "").replace("?", "").replace("!", "")
9          text = text.lower()
10         words = text.split()
11         words.sort()
12         return words
13
14
15 def get_words_dict(words):
16     words_dict = dict()
17
18     for word in words:
19         if word in words_dict:
20             words_dict[word] = words_dict[word] + 1
21         else:
22             words_dict[word] = 1
23     return words_dict
24
25
26 def main():
27     filename = input("Введите путь к файлу: ")
28     if not os.path.exists(filename):
29         print("Указанный файл не существует")
30     else:
```

```
31 words = get_words(filename)
32 words_dict = get_words_dict(words)
33 print(f"Кол-во слов: {len(words)}")
34 print(f"Кол-во уникальных слов: {len(words_dict)}")
35 print("Все использованные слова:")
36 for word in words_dict:
37     print(word.ljust(20), words_dict[word])
38
39
40 if __name__ == "__main__":
41     main()
```

Здесь в функции `get_words()` производится начальная сегментация текста на слова. При этом все пунктуационные знаки удаляются, а переводы строки заменяются на пробелы. Затем происходит разбиение текста на слова. В качестве разделителя по умолчанию применяется пробел.

Далее в функции `get_words_dict()` получаем словарь из слов, где ключ - это уникальное слово, а значение - количество вхождений данного слова в тексте.

В функции `main` осуществляется ввод пути к файлу и вызов выше определенных функций, а также вывод все статистики.

Консольный вывод программы:

```
Введите путь к файлу: C:\SomeDir\hello.txt
Кол-во слов: 66
Кол-во уникальных слов: 54
Все использованные слова:
благодетель      2
в                 1
всего             1
вы                1
горчичным        1
ее                1
ежели             3
еще               1
.....
```

[Назад](#) [Содержание](#) [Вперед](#)



[Помощь сайту](#)

[Помощь сайту](#)

Юмани:

410011174743222

Номер карты:

4048415020898850

[Телеграмм](#)

[Вконтакте](#) | [Телеграм](#) | [Донаты/Помощь сайту](#)

Contacts: metanit22@mail.ru

Copyright © Евгений Попов, metanit.com, 2025. Все права защищены.

Запись и чтение архивных zip-файлов

Последнее обновление: 24.11.2022



Zip представляет наиболее популярный формат архивации и сжатия файлов. И язык Python имеет встроенный модуль для работы с ними - **zipfile**. С помощью этого модуля можно создавать, считывать, записывать zip-файлы, получать их содержимое и добавлять в них файлы. Также поддерживается шифрование, но не поддерживается дешифрование.

Для представления zip-файла в этом модуле определен класс **ZipFile**. Он имеет следующий конструктор:

```
1 ZipFile(file, mode='r', compression=ZIP_STORED, allowZip64=True, compresslevel=None,
```

Параметры:

- **file**: путь к zip-файлу
- **mode**: режим открытия файла. Может принимать следующие значения:
 - **r**: применяется для чтения существующего файла
 - **w**: применяется для записи нового файла
 - **a**: применяется для добавления в файл
- **compression**: тип сжатия файла при записи. Может принимать значения:
 - **ZIP_STORED**: архивация без сжатия (значение по умолчанию)
 - **ZIP_DEFLATED**: стандартный тип сжатия при архивации в zip
 - **ZIP_BZIP2**: сжатие с помощью способа BZIP2
 - **ZIP_LZMA**: сжатие с помощью способа LZMA
- **allowZip64**: если равно True, то zip-файл может быть больше 4 Гб

- `compresslevel`: уровень сжатия при записи файла. Для типов сжатия `ZIP_STORED` и `ZIP_LZMA` не применяется. Для типа `ZIP_DEFLATED` допустимые значения от 0 до 9, а для типа `ZIP_BZIP2` допустимые значения от 1 до 9.
- `strict_timestamps`: при значении `False` позволяет работать с zip-файлами, созданными ранее 01.01.1980 и позже 31.12.2107
- `metadata_encoding`: применяется для декодирования метаданных zip-файла (например, комментариев)

Для работы с файлами этот класс предоставляет ряд методов:

- **`close()`**: закрывает zip-файл
- **`getinfo()`**: возвращает информацию об одном файле из архива в виде объекта `ZipInfo`
- **`namelist()`**: возвращает список файлов архива
- **`infolist()`**: возвращает информацию обо всех файлах из архива в виде списка объектов `ZipInfo`
- **`open()`**: предоставляет доступ к одному из файлов в архиве
- **`read()`**: считывает файл из архива в набор байтов
- **`extract()`**: извлекает из архива один файл
- **`extractall()`**: извлекает все элементы из архива
- **`setpassword()`**: устанавливает пароль для zip-файла
- **`printdir()`**: выводит на консоль содержимое архива

Создание и закрытие файла

Для создания архивного файла в конструктор `ZipFile` передается режим `"w"` или `"a"`:

```
1 from zipfile import ZipFile
2
3 myzip = ZipFile("metanit.zip", "w")
```

После выполнения кода в текущей папке будет создаваться пустой архивный файл `"metanit.zip"`.

После окончания работы с архивом для его закрытия применяется метод `close()`:

```
1 from zipfile import ZipFile
2
```

```
3 myzip = ZipFile("metanit.zip", "w")
4 myzip.close()
```

Но так как ZipFile также представляет менеджер контекста, то он поддерживает выражение **with**, которое определяет контекст и автоматически закрывает файл по завершению контекста:

```
1 from zipfile import ZipFile
2
3 with ZipFile("metanit.zip", "w") as myzip:
4     pass
```

Запись файлов в архив

Для записи файлов в архив применяется файл **write()**:

```
1 write(filename, arcname=None, compress_type=None, compresslevel=None)
```

Первый параметр представляет файл, который записывается в архив. Второй параметр - `arcname` устанавливает произвольное имя для файла внутри архива (по умолчанию это само имя файла). Третий параметр - `compress_type` представляет тип сжатия, а параметр `compresslevel` - уровень сжатия.

Например, запишем в архив "metanit.zip" файл "hello.txt" (который, как предполагается, находится в той же папке, где и текущий скрипт python):

```
1 from zipfile import ZipFile
2
3 with ZipFile("metanit.zip", "w") as myzip:
4     myzip.write("hello.txt")
```

Стоит учитывать, что при открытии файла в режиме "w" при всех последующих записях текущее содержимое будет затираться, то есть фактически архивный файл будет создаваться заново. Если нам необходимо добавить, то необходимо определять zip-файл в режиме "a":

```
1 from zipfile import ZipFile
2
3 with ZipFile("metanit.zip", "a") as myzip:
4     myzip.write("hello2.txt")
5     myzip.write("forest.jpg")
```

Стоит отметить, что по умолчанию сжатие не применяется. Но при необходимости можно применить какой-нибудь способ сжатия и уровень сжатия"

```
1 from zipfile import ZipFile, ZIP_DEFLATED
2
3 with ZipFile("metanit.zip", "w", compression=ZIP_DEFLATED, compresslevel=3) as myzip:
4     myzip.write("hello.txt")
```

Необходимо учитывать, что если мы попробуем добавить в архив файлы с уже имеющимися именами, то консоль выведет предупреждение. Чтобы избежать наличия файлов с дублирующимися именами можно через второй папарметр метода write явным образом определить для них уникальное имя внутри архива:

```
1 from zipfile import ZipFile
2
3 with ZipFile("metanit.zip", "a") as myzip:
4     myzip.write("hello.txt", "hello1.txt")
5     myzip.write("hello.txt", "hello2.txt")
6     myzip.write("hello.txt", "hello3.txt")
```

Получение информации о файлах в архиве

Метод **infolist()** возвращает информацию о файлах в архиве с виде списка, где каждый отдельный файл представлен объектом ZipInfo:

```
1 from zipfile import ZipFile
2
3 with ZipFile("metanit.zip", "a") as myzip:
4     print(myzip.infolist())
```

Класс **ZipInfo** предоставляет ряд атрибутов для хранения информации о файле. Основные из них:

- **filename**: название файла
- **date_time**: дата и время последнего изменения файла в виде кортежа в формате (год, месяц, день, час, минута, секунда)
- **compress_type**: тип сжатия
- **compress_size**: размер после сжатия
- **file_size**: оригинальный размер файла до сжатия

Получим эти данные по каждому отдельному файлу в архиве:

```
1 from zipfile import ZipFile
2
3 with ZipFile("metanit.zip", "r") as myzip:
4     for item in myzip.infolist():
5         print(f"File Name: {item.filename} Date: {item.date_time} Size: {item.file_s
```

Примерный консольный вывод:


```
File Name: hello.txt   Date: (2022, 11, 23, 20, 21, 34)   Size: 18
File Name: forest.jpg  Date: (2022, 11, 19, 20, 46, 52)   Size: 103956
File Name: hello1.txt   Date: (2022, 11, 23, 20, 21, 34)   Size: 18
File Name: hello2.txt   Date: (2022, 11, 23, 20, 21, 34)   Size: 18
File Name: hello3.txt   Date: (2022, 11, 23, 20, 21, 34)   Size: 18
```

С помощью метода `is_dir()` можно проверить, является ли элемент в архиве папкой:

```
1 from zipfile import ZipFile
2
3 with ZipFile("metanit.zip", "r") as myzip:
4     for item in myzip.infolist():
5         if(item.is_dir()):
6             print(f"Папка: {item.filename}")
7         else:
8             print(f"Файл: {item.filename}")
```

Если надо получить только список имен входящих в архив файлов, то применяется метод **`namelist()`**:

```
1 from zipfile import ZipFile
2
3 with ZipFile("metanit.zip", "r") as myzip:
4     for item in myzip.namelist():
5         print(item)
```

Консольный вывод в моем случае:

```
hello.txt
forest.jpg
hello1.txt
hello2.txt
hello3.txt
```

С помощью метода **`getinfo()`** можно получить данные по одному из архивированных файлов, передав в метод его имя в архиве. Результат метода - объект `ZipInfo`:

```
1 from zipfile import ZipFile
2
3 with ZipFile("metanit.zip", "r") as myzip:
4     try:
5         hello_file = myzip.getinfo("hello.txt")
6         print(hello_file.file_size)
7     except KeyError:
8         print("Указанный файл отсутствует")
```

Если в архиве не окажется элемента с указанным именем, то метод сгенерирует ошибку `KeyError`.

Извлечение файлов из архива

Для извлечения всех файлов из архива применяется метод **`extractall()`**:

```
1 extractall(path=None, members=None, pwd=None)
```

Первый параметр метода устанавливает каталог для извлечения архива (по умолчанию извлечение идет в текущий каталог). Параметр `members` представляет список строк - список названий файлов, которые надо извлечь из архива. И третий параметр - `pwd` представляет пароль, в случае если архив закрыт паролем.

Например, извлечем все файлы из архива:

```
1 from zipfile import ZipFile
2
3 with ZipFile("metanit.zip", "r") as myzip:
4     myzip.extractall()
```

Извлечение в определенную папку:

```
1 myzip.extractall(path="metanit")
```

Извлечение части файлов:

```
1 # извлекаем файлы "hello.txt", "forest.jpg" в папку "metanit2"
2 myzip.extractall(path="metanit2", members=["hello.txt", "forest.jpg"])
```

Для извлечения одного файла применяется метод **`extract()`**, в который в качестве обязательного параметра передается имя извлекаемого файла:

```
1 myzip.extract("hello.txt")
```

Считывание файла

Метод **`read()`** позволяет считать содержимое файла из архива в набор байтов:

```
1 from zipfile import ZipFile
2
3 with ZipFile("metanit.zip", "r") as myzip:
4     content = myzip.read("hello5.txt")
5     print(content)
```

Открытие файла

Метод **`open()`** позволяет открывать отдельные файлы из архива без непосредственного их извлечения:

```
1 open(name, mode='r', pwd=None, *, force_zip64=False)
```

В качестве первого обязательного параметра передается имя файла внутри архива. Второй параметр - mode устанавливает режим открытия. Параметр pwd задает пароль, если файл защищен паролем. И параметр force_zip64 при значении True позволяет открывать файлы больше 4 Гб.

Этот файл может быть полезен для манипулирования файлом, например, для считывания его содержимого или, наоборот, для записи в него. Например, откроем файл и считаем его содержимое:

```
1 from zipfile import ZipFile
2
3 with ZipFile("metanit.zip", "a") as myzip:
4     # записываем в архив новый файл "hello5.txt"
5     with myzip.open("hello5.txt", "w") as hello_file:
6         encoded_str = bytes("Python...", "UTF-8")
7         hello_file.write(encoded_str)
```

[Назад](#) [Содержание](#) [Вперед](#)



Помощь сайту

[Помощь сайту](#)

Юмани:

410011174743222

Номер карты:

4048415020898850

[Телеграмм](#)

[Вконтакте](#) | [Телеграм](#) | [Донаты/Помощь сайту](#)

Contacts: metanit22@mail.ru

Copyright © Евгений Попов, metanit.com, 2025. Все права защищены.